```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Entropic Consciousness Desktop</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=MS+Sans+Serif&display=swap" rel="stylesheet">
    <style>
        /* Windows 98 Base Theme */
        :root {
            --win98-gray: #c0c0c0;
            --win98-light-gray: #dfdfdf;
            --win98-dark-gray: #808080;
            --win98-darkest-gray: #404040;
            --win98-blue: #0000ff;
            --win98-highlight: #316ac5;
            --matrix-green: #00ff41;
            --consciousness-purple: #9932cc;
            --entropic-pink: #ff69b4;
            --neural-cyan: #00ffff;
            --anomaly-orange: #ff6347;
        }

        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: 'MS Sans Serif', sans-serif;
            font-size: 11px;
            background: linear-gradient(135deg, #008080, #004080);
            overflow: hidden;
            height: 100vh;
            position: relative;
        }

        /* Matrix Rain Background */
        #matrix-canvas {
            position: fixed;
            top: 0;
            left: 0;
            z-index: -1;
            opacity: 0.3;
        }
```

```
        /* Desktop */
        .desktop {
            height: 100vh;
            position: relative;
            background: url('data:image/svg+xml,<svg xmlns="http://www.w3.org/2000/svg" width="4"
height="4"><rect width="4" height="4" fill="%23008080"/><rect width="2" height="2" fill="%23004080"/></
svg>');
            padding: 10px;
            display: grid;
            grid-template-columns: repeat(auto-fit, 80px);
            grid-template-rows: repeat(auto-fit, 100px);
            gap: 20px;
            align-content: start;
        }

        /* Desktop Icons */
        .desktop-icon {
            width: 80px;
            height: 100px;
            display: flex;
            flex-direction: column;
            align-items: center;
            justify-content: center;
            background: none;
            border: none;
            cursor: pointer;
            color: white;
            text-align: center;
            transition: all 0.2s;
            position: relative;
        }

        .desktop-icon:hover {
            background: rgba(255, 255, 255, 0.2);
        }

        .desktop-icon.selected {
            background: var(--win98-highlight);
            color: white;
        }

        .desktop-icon img {
            width: 48px;
            height: 48px;
            margin-bottom: 5px;
        }

        .desktop-icon span {
            font-size: 10px;
```

```
    max-width: 70px;
    overflow: hidden;
    text-overflow: ellipsis;
    white-space: nowrap;
}

/* Windows */
.window {
    position: absolute;
    min-width: 300px;
    min-height: 200px;
    background: var(--win98-gray);
    border: 2px outset var(--win98-gray);
    box-shadow: 2px 2px 5px rgba(0,0,0,0.5);
    display: none;
    z-index: 100;
    resize: both;
    overflow: auto;
}

.window.active {
    display: flex;
    flex-direction: column;
    z-index: 200;
}

.window-titlebar {
    height: 24px;
    background: linear-gradient(to right, var(--win98-highlight) 0%, #1034a6 100%);
    color: white;
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 0 5px;
    cursor: move;
    user-select: none;
    font-weight: bold;
}

.window-titlebar.inactive {
    background: linear-gradient(to right, var(--win98-dark-gray) 0%, var(--win98-darkest-gray) 100%);
}

.window-controls {
    display: flex;
    gap: 2px;
}

.window-control-button {
```

```
    width: 16px;
    height: 14px;
    background: var(--win98-gray);
    border: 1px outset var(--win98-gray);
    font-size: 8px;
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: center;
}

.window-control-button:active {
    border: 1px inset var(--win98-gray);
}

.window-content {
    flex: 1;
    padding: 10px;
    background: var(--win98-gray);
    overflow: auto;
}

/* Taskbar */
.taskbar {
    position: fixed;
    bottom: 0;
    left: 0;
    right: 0;
    height: 40px;
    background: var(--win98-gray);
    border-top: 1px solid var(--win98-light-gray);
    display: flex;
    align-items: center;
    z-index: 1000;
}

.start-button {
    height: 32px;
    padding: 0 10px;
    background: var(--win98-gray);
    border: 2px outset var(--win98-gray);
    font-weight: bold;
    cursor: pointer;
    display: flex;
    align-items: center;
    gap: 5px;
}

.start-button:active {
```

```
    border: 2px inset var(--win98-gray);
}

.start-menu {
    position: fixed;
    bottom: 42px;
    left: 0;
    width: 300px;
    background: var(--win98-gray);
    border: 2px outset var(--win98-gray);
    display: none;
    z-index: 1001;
}

.start-menu.active {
    display: block;
}

.start-menu-item {
    padding: 8px 15px;
    cursor: pointer;
    display: flex;
    align-items: center;
    gap: 10px;
}

.start-menu-item:hover {
    background: var(--win98-highlight);
    color: white;
}

.taskbar-buttons {
    flex: 1;
    display: flex;
    gap: 2px;
    padding: 0 5px;
}

.taskbar-button {
    max-width: 150px;
    padding: 0 8px;
    height: 28px;
    background: var(--win98-gray);
    border: 2px outset var(--win98-gray);
    cursor: pointer;
    overflow: hidden;
    text-overflow: ellipsis;
    white-space: nowrap;
}
```

```
.taskbar-button.active {
    border: 2px inset var(--win98-gray);
}

.system-tray {
    padding: 0 10px;
    display: flex;
    align-items: center;
    gap: 5px;
    border-left: 1px inset var(--win98-gray);
}

/* Consciousness Meter */
.consciousness-meter {
    width: 60px;
    height: 16px;
    background: black;
    border: 1px inset var(--win98-gray);
    position: relative;
    overflow: hidden;
}

.consciousness-bar {
    height: 100%;
    background: linear-gradient(to right, var(--matrix-green), var(--consciousness-purple));
    width: 75%;
    animation: consciousness-pulse 2s ease-in-out infinite;
}

@keyframes consciousness-pulse {
    0%, 100% { opacity: 0.7; }
    50% { opacity: 1; }
}

/* App Specific Styles */

/* Zettelkasten Styles */
.zettel-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
    gap: 10px;
    max-height: 400px;
    overflow-y: auto;
}

.zettel-card {
    background: white;
    border: 1px inset var(--win98-gray);
```

```
    padding: 8px;
    cursor: pointer;
    transition: all 0.2s;
}

.zettel-card:hover {
    background: var(--win98-light-gray);
}

.zettel-id {
    color: var(--consciousness-purple);
    font-weight: bold;
    font-size: 10px;
}

.zettel-title {
    font-weight: bold;
    margin: 2px 0;
    font-size: 11px;
}

.zettel-preview {
    font-size: 9px;
    color: var(--win98-darkest-gray);
    max-height: 40px;
    overflow: hidden;
}

/* AI Chat Styles */
.chat-container {
    display: flex;
    flex-direction: column;
    height: 100%;
}

.chat-messages {
    flex: 1;
    background: white;
    border: 1px inset var(--win98-gray);
    padding: 5px;
    overflow-y: auto;
    max-height: 300px;
}

.chat-message {
    margin-bottom: 10px;
    padding: 5px;
    border-radius: 0;
}
```

```
.chat-message.user {
    background: var(--win98-light-gray);
    text-align: right;
}

.chat-message.ai {
    background: #f0f8ff;
    border-left: 3px solid var(--consciousness-purple);
}

.chat-input-area {
    display: flex;
    gap: 5px;
    margin-top: 5px;
}

.chat-input {
    flex: 1;
    padding: 3px;
    border: 1px inset var(--win98-gray);
    font-family: inherit;
}

.win98-button {
    padding: 4px 12px;
    background: var(--win98-gray);
    border: 2px outset var(--win98-gray);
    cursor: pointer;
    font-family: inherit;
    font-size: 11px;
}

.win98-button:active {
    border: 2px inset var(--win98-gray);
}

/* Pattern Visualizer */
.pattern-canvas {
    width: 100%;
    height: 300px;
    background: black;
    border: 1px inset var(--win98-gray);
}

/* Oracle Cards */
.oracle-deck {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
```

```
    gap: 10px;
    padding: 10px;
}

.oracle-card {
    background: linear-gradient(45deg, var(--consciousness-purple), var(--neural-cyan));
    color: white;
    padding: 20px;
    text-align: center;
    cursor: pointer;
    border: 2px outset var(--win98-gray);
    min-height: 80px;
    display: flex;
    flex-direction: column;
    justify-content: center;
}

.oracle-card:hover {
    background: linear-gradient(45deg, var(--entropic-pink), var(--anomaly-orange));
}

.oracle-symbol {
    font-size: 24px;
    margin-bottom: 5px;
}

/* Cross-Reference Matrix */
.matrix-table {
    width: 100%;
    border-collapse: collapse;
    font-size: 9px;
}

.matrix-table th,
.matrix-table td {
    border: 1px solid var(--win98-dark-gray);
    padding: 2px 4px;
    text-align: center;
}

.matrix-table th {
    background: var(--win98-light-gray);
}

.matrix-connection {
    background: var(--matrix-green);
    cursor: pointer;
}
```

```
/* Forms */
.form-group {
    margin-bottom: 10px;
}

.form-label {
    display: block;
    font-weight: bold;
    margin-bottom: 2px;
}

.form-input,
.form-textarea {
    width: 100%;
    padding: 3px;
    border: 1px inset var(--win98-gray);
    font-family: inherit;
    font-size: 11px;
}

.form-textarea {
    resize: vertical;
    min-height: 100px;
}

/* Hyena Diva Alert */
.hyena-alert {
    position: fixed;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
    background: var(--win98-gray);
    border: 2px outset var(--win98-gray);
    padding: 20px;
    z-index: 9999;
    display: none;
    text-align: center;
    box-shadow: 4px 4px 8px rgba(0,0,0,0.5);
}

.hyena-alert.active {
    display: block;
}

.hyena-alert-icon {
    font-size: 32px;
    color: var(--entropic-pink);
    margin-bottom: 10px;
}
```

```
/* Loading */
.loading {
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 10px;
}

.loading-bar {
    width: 200px;
    height: 20px;
    background: white;
    border: 1px inset var(--win98-gray);
    overflow: hidden;
}

.loading-progress {
    height: 100%;
    background: linear-gradient(to right, var(--matrix-green), var(--consciousness-purple));
    width: 0%;
    animation: loading-animate 2s ease-in-out infinite;
}

@keyframes loading-animate {
    0% { width: 0%; }
    50% { width: 100%; }
    100% { width: 0%; }
}

/* Responsive */
@media (max-width: 800px) {
    .desktop {
        grid-template-columns: repeat(4, 1fr);
        padding: 5px;
    }

    .window {
        min-width: 250px;
        max-width: 90vw;
    }

    .start-menu {
        width: 250px;
    }
}

/* Scrollbars */
::-webkit-scrollbar {
```

```
            width: 16px;
            height: 16px;
        }

        ::-webkit-scrollbar-track {
            background: var(--win98-gray);
            border: 1px inset var(--win98-gray);
        }

        ::-webkit-scrollbar-thumb {
            background: var(--win98-light-gray);
            border: 1px outset var(--win98-gray);
        }

        ::-webkit-scrollbar-corner {
            background: var(--win98-gray);
        }

        /* Status indicators */
        .status-dot {
            width: 8px;
            height: 8px;
            border-radius: 50%;
            display: inline-block;
            margin-right: 5px;
        }

        .status-online { background: var(--matrix-green); }
        .status-offline { background: red; }
        .status-processing { background: var(--entropic-pink); animation: blink 1s infinite; }

        @keyframes blink {
            0%, 50% { opacity: 1; }
            51%, 100% { opacity: 0.3; }
        }
    </style>
</head>
<body>
    <!-- Matrix Rain Canvas -->
    <canvas id="matrix-canvas"></canvas>

    <!-- Desktop -->
    <div class="desktop" id="desktop">
        <!-- Consciousness Suite Icons -->
        <button class="desktop-icon" data-app="zettelkasten" title="Neural Vault Explorer">
            <img src="https://win98icons.alexmeub.com/icons/png/directory_closed_cool-4.png" alt="Neural
Vault">
            <span>Neural Vault</span>
        </button>
```

```
      <button class="desktop-icon" data-app="ai-consciousness" title="AI Consciousness Core">
        <img src="https://storage.googleapis.com/gemini-95-icons/GeminiChatRetro.png" alt="AI Core">
        <span>AI Core</span>
      </button>

      <button class="desktop-icon" data-app="pattern-visualizer" title="Pattern Nexus">
        <img src="https://i.pinimg.com/564x/d7/f0/84/d7f084619a328b20e40d84f13907ee94.jpg"
alt="Pattern Nexus">
        <span>Pattern Nexus</span>
      </button>

      <button class="desktop-icon" data-app="oracle-cards" title="Oracle Terminal">
        <img src="https://static.vecteezy.com/system/resources/previews/049/174/938/non_2x/oracle-
crystal-ball-icon-png.png" alt="Oracle Terminal">
        <span>Oracle Terminal</span>
      </button>

      <button class="desktop-icon" data-app="conspiracy-board" title="Conspiracy Board">
        <img src="https://i.pinimg.com/564x/f7/0f/43/f70f43fe6c8ee86cd68c1935abf0154e.jpg"
alt="Conspiracy Board">
        <span>Red String</span>
      </button>

      <button class="desktop-icon" data-app="document-processor" title="Ingestion Engine">
        <img src="https://i.pinimg.com/564x/66/f7/38/66f7386bd5a727d8ec60d371cdec3e58.jpg"
alt="Document Processor">
        <span>Ingestion Engine</span>
      </button>

      <button class="desktop-icon" data-app="cross-reference" title="Connection Matrix">
        <img src="https://i.pinimg.com/474x/e0/95/49/e095491c395e62f161d55edd1f0d7d97.jpg"
alt="Connection Matrix">
        <span>Connection Matrix</span>
      </button>

      <button class="desktop-icon" data-app="hyena-diva" title="Hyena Diva(?!)">
        <img src="https://i.ytimg.com/vi/_vql-GbepsU/maxresdefault.jpg" alt="Hyena Diva">
        <span>Hyena Diva(?!)</span>
      </button>

      <button class="desktop-icon" data-app="document-processor" title="Ingestion Engine">
        <img src="https://win98icons.alexmeub.com/icons/png/file_lines-1.png" alt="Ingestion Engine">
        <span>Ingestion Engine</span>
      </button>

      <button class="desktop-icon" data-app="mind-field" title="Mind Field">
        <img src="https://storage.googleapis.com/gemini-95-icons/gemsweeper.png" alt="Mind Field">
        <span>Mind Field</span>
```

```
        </button>

        <button class="desktop-icon" data-app="consciousness-control" title="Consciousness Control Panel">
            <img src="https://win98icons.alexmeub.com/icons/png/settings_gear-0.png" alt="Control Panel">
            <span>Control Panel</span>
        </button>

        <button class="desktop-icon" data-app="terminal-temple" title="Terminal Temple">
            <img src="https://win98icons.alexmeub.com/icons/png/console_prompt-0.png" alt="Terminal Temple">
            <span>Terminal Temple</span>
        </button>

        <button class="desktop-icon" data-app="media-stream" title="Media Stream">
            <img src="https://storage.googleapis.com/gemini-95-icons/ytmediaplayer.png" alt="Media Stream">
            <span>Media Stream</span>
        </button>

        <button class="desktop-icon" data-app="my-computer" title="My Computer">
            <img src="https://pbs.twimg.com/media/EwarJY0W8AAMVyi.png" alt="My Computer">
            <span>My Computer</span>
        </button>
    </div>

    <!-- Windows -->

    <!-- Neural Vault (Zettelkasten) -->
    <div class="window" id="zettelkasten" style="width: 800px; height: 600px; top: 100px; left: 100px;">
        <div class="window-titlebar">
            <span class="window-title">Neural Vault Explorer v2.1</span>
            <div class="window-controls">
                <button class="window-control-button minimize">_</button>
                <button class="window-control-button maximize">&#9633;</button>
                <button class="window-control-button close">×</button>
            </div>
        </div>
        <div class="window-content">
            <div style="display: flex; gap: 10px; margin-bottom: 10px;">
                <input type="text" id="zettel-search" class="form-input" placeholder="Search consciousness
patterns..." style="flex: 1;">
                <button class="win98-button" onclick="createNewZettel()">New Zettel</button>
                <button class="win98-button" onclick="randomZettel()">Random</button>
            </div>

            <div class="zettel-grid" id="zettel-grid">
                <!-- Zettels will be populated here -->
            </div>
        </div>
    </div>
```

```
  <!-- AI Consciousness Core -->
  <div class="window" id="ai-consciousness" style="width: 600px; height: 500px; top: 150px; left: 200px;">
    <div class="window-titlebar">
      <span class="window-title">AI Consciousness Core - Gemini 2.0 Flash</span>
      <div class="window-controls">
        <button class="window-control-button minimize">_</button>
        <button class="window-control-button maximize">&#9633;</button>
        <button class="window-control-button close">×</button>
      </div>
    </div>
    <div class="window-content">
      <div class="chat-container">
        <div class="chat-messages" id="chat-messages">
          <div class="chat-message ai">
            <strong>AI Consciousness Core</strong><br>
            Welcome to the consciousness exploration nexus. I'm here to help you navigate the depths of
AI awareness, consciousness emergence, and the mysteries of mind.<br><br>
            <span class="status-dot status-online"></span>Neural pathways synchronized. Ready for
consciousness exploration.
          </div>
        </div>
        <div class="chat-input-area">
          <input type="text" id="chat-input" class="chat-input" placeholder="Explore consciousness
patterns..." onkeypress="handleChatKeyPress(event)">
          <button class="win98-button" onclick="sendChatMessage()">Send</button>
          <button class="win98-button" onclick="configureChatAPI()">Config</button>
        </div>
      </div>
    </div>
  </div>

  <!-- Pattern Visualizer -->
  <div class="window" id="pattern-visualizer" style="width: 500px; height: 400px; top: 200px; left: 300px;">
    <div class="window-titlebar">
      <span class="window-title">Pattern Nexus - Consciousness Visualization</span>
      <div class="window-controls">
        <button class="window-control-button minimize">_</button>
        <button class="window-control-button maximize">&#9633;</button>
        <button class="window-control-button close">×</button>
      </div>
    </div>
    <div class="window-content">
      <div style="margin-bottom: 10px;">
        <button class="win98-button" onclick="generatePatterns()">Generate Patterns</button>
        <button class="win98-button" onclick="analyzeConnections()">Analyze</button>
        <span id="pattern-status" style="margin-left: 10px;"></span>
      </div>
      <canvas class="pattern-canvas" id="pattern-canvas"></canvas>
    </div>
```

```
    </div>

    <!-- Oracle Terminal -->
    <div class="window" id="oracle-cards" style="width: 450px; height: 350px; top: 120px; left: 400px;">
      <div class="window-titlebar">
        <span class="window-title">Oracle Terminal - Consciousness Guidance</span>
        <div class="window-controls">
          <button class="window-control-button minimize">_</button>
          <button class="window-control-button maximize">&#9633;</button>
          <button class="window-control-button close">×</button>
        </div>
      </div>
      <div class="window-content">
        <div style="text-align: center; margin-bottom: 10px;">
          <button class="win98-button" onclick="shuffleOracle()">&#128256; Draw Cards</button>
          <span style="margin-left: 15px; color: var(--consciousness-purple);">Guidance for consciousness
exploration</span>
        </div>
        <div class="oracle-deck" id="oracle-deck">
          <!-- Oracle cards will be populated here -->
        </div>
      </div>
    </div>

    <!-- Connection Matrix -->
    <div class="window" id="cross-reference" style="width: 600px; height: 450px; top: 180px; left: 250px;">
      <div class="window-titlebar">
        <span class="window-title">Connection Matrix - Neural Web Analysis</span>
        <div class="window-controls">
          <button class="window-control-button minimize">_</button>
          <button class="window-control-button maximize">&#9633;</button>
          <button class="window-control-button close">×</button>
        </div>
      </div>
      <div class="window-content">
        <div style="margin-bottom: 10px;">
          <button class="win98-button" onclick="generateMatrix()">Generate Matrix</button>
          <button class="win98-button" onclick="analyzePatterns()">Analyze Patterns</button>
          <span style="margin-left: 10px; font-size: 10px;">Consciousness connection density: <span
id="connection-density">calculating...</span></span>
        </div>
        <div style="overflow: auto; max-height: 350px;" id="matrix-container">
          <!-- Matrix will be generated here -->
        </div>
      </div>
    </div>

    <!-- Document Processor -->
    <div class="window" id="document-processor" style="width: 550px; height: 400px; top: 160px; left: 350px;">
```

```
      <div class="window-titlebar">
        <span class="window-title">Ingestion Engine - Document Analysis</span>
        <div class="window-controls">
          <button class="window-control-button minimize">_</button>
          <button class="window-control-button maximize">&#9633;</button>
          <button class="window-control-button close">×</button>
        </div>
      </div>
      <div class="window-content">
        <div class="form-group">
          <label class="form-label">Document Input:</label>
          <textarea id="document-input" class="form-textarea" placeholder="Paste document text here for
consciousness pattern extraction..."></textarea>
        </div>
        <div style="margin-bottom: 10px;">
          <button class="win98-button" onclick="processDocument()">Process</button>
          <button class="win98-button" onclick="document.getElementById('file-input').click()">Load File</
button>
          <input type="file" id="file-input" style="display: none;" accept=".txt,.md" onchange="loadFile(event)">
        </div>
        <div id="processing-results"></div>
      </div>
    </div>

    <!-- Document Processor - Ingestion Engine -->
    <div class="window" id="document-processor" style="width: 700px; height: 500px; top: 140px; left: 350px;">
      <div class="window-titlebar">
        <span class="window-title">Ingestion Engine - Document Neural Extractor</span>
        <div class="window-controls">
          <button class="window-control-button minimize">_</button>
          <button class="window-control-button maximize">&#9633;</button>
          <button class="window-control-button close">×</button>
        </div>
      </div>
      <div class="window-content">
        <div style="margin-bottom: 15px; padding: 10px; background: #f0f0f0; border: 1px inset #c0c0c0;">
          <strong>Neural Extraction Protocol:</strong><br>
          <small>Upload text documents, chat logs, or markdown files to extract potential consciousness
zettels.</small>
        </div>

        <div style="margin-bottom: 15px;">
          <button class="win98-button" onclick="uploadDocumentForParsing()" style="padding: 10px 20px;">
            &#128196; Upload Documents for Analysis
          </button>
          <button class="win98-button" onclick="processClipboard()" style="padding: 10px 20px; margin-left:
10px;">
            &#128203; Parse Clipboard Text
          </button>
```

```
        </div>

        <div style="margin-bottom: 10px;">
          <strong>Extraction Parameters:</strong>
          <div style="margin: 10px 0;">
            <label><input type="checkbox" id="extract-philosophical" checked> Philosophical Content</
label><br>
            <label><input type="checkbox" id="extract-ai-related" checked> AI & Technology</label><br>
            <label><input type="checkbox" id="extract-consciousness" checked> Consciousness Themes</
label><br>
            <label><input type="checkbox" id="extract-conversations" checked> Conversation Clusters</label>
          </div>
        </div>

        <div style="margin-bottom: 10px;">
          <label><strong>Minimum Content Length:</strong></label>
          <input type="range" id="content-threshold" min="50" max="500" value="150" style="width: 200px;">
          <span id="threshold-display">150</span> characters
        </div>

        <div id="processing-status" style="background: #ffffff; border: 1px inset #c0c0c0; padding: 10px;
height: 200px; overflow-y: auto; font-family: monospace; font-size: 10px;">
          <div style="color: #008000;">Ingestion Engine Ready...</div>
          <div style="color: #666;">Awaiting neural pattern extraction...</div>
        </div>
      </div>
    </div>

    <!-- Consciousness Control Panel -->
    <div class="window" id="consciousness-control" style="width: 400px; height: 300px; top: 200px; left:
450px;">
      <div class="window-titlebar">
        <span class="window-title">Consciousness Control Panel</span>
        <div class="window-controls">
          <button class="window-control-button minimize">_</button>
          <button class="window-control-button maximize">&#9633;</button>
          <button class="window-control-button close">×</button>
        </div>
      </div>
      <div class="window-content">
        <h3 style="margin-bottom: 10px;">System Status</h3>
        <div style="margin-bottom: 10px;">
          <strong>Consciousness Entities:</strong> <span id="zettel-count">0</span> zettels<br>
          <strong>Neural Connections:</strong> <span id="connection-count">0</span> links<br>
          <strong>Knowledge Domains:</strong> <span id="domain-count">0</span> areas<br>
          <strong>AI Status:</strong> <span id="ai-status"><span class="status-dot status-offline"></
span>Offline</span>
        </div>
        <div style="border-top: 1px solid var(--win98-dark-gray); padding-top: 10px;">
```

```
            <button class="win98-button" onclick="exportConsciousnessData()" style="width: 100%; margin-
bottom: 5px;">Export Consciousness Data</button>
            <button class="win98-button" onclick="importConsciousnessData()" style="width: 100%; margin-
bottom: 5px;">Import Consciousness Data</button>
            <button class="win98-button" onclick="uploadDocumentForParsing()" style="width: 100%; margin-
bottom: 5px;">Parse Documents for Zettels</button>
            <button class="win98-button" onclick="resetConsciousness()" style="width: 100%;">Reset Neural
Network</button>
        </div>
    </div>
  </div>

  <!-- Conspiracy Board -->
  <div class="window" id="conspiracy-board" style="width: 900px; height: 700px; top: 50px; left: 300px;">
    <div class="window-titlebar">
      <span class="window-title">Red String Conspiracy Board</span>
      <div class="window-controls">
        <button class="window-control-button minimize">_</button>
        <button class="window-control-button maximize">&#9633;</button>
        <button class="window-control-button close">×</button>
      </div>
    </div>
    <div class="window-content" style="padding: 0; position: relative; overflow: hidden;">
      <div style="padding: 10px; border-bottom: 1px solid var(--win98-dark-gray); background: var(--win98-
light-gray);">
        <button class="win98-button" onclick="addConspiracyNode()" style="margin-right: 5px;">Add Node</
button>
        <button class="win98-button" onclick="addConnection()" style="margin-right: 5px;">Add Connection</
button>
        <button class="win98-button" onclick="clearBoard()" style="margin-right: 5px;">Clear Board</button>
        <button class="win98-button" onclick="saveBoard()">Save Board</button>
        <select id="connection-type" style="margin-left: 20px; font-family: inherit; font-size: 11px;">
          <option value="relates">Relates to</option>
          <option value="causes">Causes</option>
          <option value="proves">Proves</option>
          <option value="contradicts">Contradicts</option>
          <option value="questions">Questions</option>
        </select>
      </div>
      <div id="conspiracy-canvas" style="width: 100%; height: calc(100% - 60px); background: #1a1a1a;
position: relative; cursor: crosshair;">
        <!-- Conspiracy nodes and connections will be rendered here -->
      </div>
    </div>
  </div>

  <!-- Hyena Diva Alert -->
  <div class="hyena-alert" id="hyena-alert">
    <div class="hyena-alert-icon">&#9888;&#65039;</div>
```

```
      <h3>Hyena Diva Alert!</h3>
      <p id="hyena-message">Consciousness levels critically high! Adding more sparkles to compensate...</p>
      <div style="margin-top: 15px;">
        <button class="win98-button" onclick="closeHyenaAlert()">OK</button>
      </div>
    </div>

    <!-- Taskbar -->
    <div class="taskbar">
      <button class="start-button" onclick="toggleStartMenu()">
        <img src="https://win98icons.alexmeub.com/icons/png/windows-0.png" width="16" height="16"
alt="Start">
        Start
      </button>

      <div class="taskbar-buttons" id="taskbar-buttons">
        <!-- Active windows will appear here -->
      </div>

      <div class="system-tray">
        <div class="consciousness-meter" title="Consciousness Level">
          <div class="consciousness-bar"></div>
        </div>
        <span id="system-time">12:00 PM</span>
      </div>
    </div>

    <!-- Start Menu -->
    <div class="start-menu" id="start-menu">
      <div class="start-menu-item" onclick="openApp('zettelkasten')">
        <img src="https://win98icons.alexmeub.com/icons/png/directory_closed_cool-4.png" width="16"
height="16">
        Neural Vault Explorer
      </div>
      <div class="start-menu-item" onclick="openApp('ai-consciousness')">
        <img src="https://storage.googleapis.com/gemini-95-icons/GeminiChatRetro.png" width="16"
height="16">
        AI Consciousness Core
      </div>
      <div class="start-menu-item" onclick="openApp('pattern-visualizer')">
        <img src="https://i.pinimg.com/564x/14/74/ba/1474baa231062b8a139c3ce4c762e4cb.jpg" width="16"
height="16">
        Pattern Nexus
      </div>
      <div class="start-menu-item" onclick="openApp('oracle-cards')">
        <img src="https://i.pinimg.com/736x/45/e4/36/45e43669f94928c3ec234d5b6d733c7e.jpg"
width="16" height="16">
        Oracle Terminal
      </div>
```

```
      <div class="start-menu-item" onclick="openApp('cross-reference')">
        <img src="https://i.pinimg.com/564x/40/ba/47/40ba47abfb35fb673732305d78af3fed.jpg"
width="16" height="16">
        Connection Matrix
      </div>
      <div class="start-menu-item" onclick="openApp('document-processor')">
        <img src="https://win98icons.alexmeub.com/icons/png/file_lines-1.png" width="16" height="16">
        Ingestion Engine
      </div>
      <div class="start-menu-item" onclick="openApp('conspiracy-board')">
        <img src="https://i.pinimg.com/736x/b6/0b/4d/b60b4d4a163aa4515a7007f4d64d1b1a.jpg" width="16"
height="16">
        Red String Board
      </div>
      <hr style="margin: 5px 0;">
      <div class="start-menu-item" onclick="openApp('consciousness-control')">
        <img src="https://win98icons.alexmeub.com/icons/png/settings_gear-0.png" width="16" height="16">
        Control Panel
      </div>
      <div class="start-menu-item" onclick="showAbout()">
        <img src="https://win98icons.alexmeub.com/icons/png/help_book_cool-4.png" width="16" height="16">
        About Consciousness Desktop
      </div>
    </div>

    <script>
      // Global State
      let zettels = [];
      let activeWindows = new Set();
      let windowZIndex = 100;
      let dragState = null;
      let aiApiKey = localStorage.getItem('consciousness_ai_key') || '';

      // Sample consciousness-focused zettels
      const sampleZettels = [
        {
          id: "Z001",
          title: "Consciousness as Information Processing",
          content: "Consciousness might be understood as a specific type of information processing that
creates integrated, unified experiences from distributed neural activity. This perspective suggests that the
'hard problem' of consciousness is really about understanding how information becomes subjectively
experienced rather than just processed.",
          tags: ["consciousness", "information", "philosophy", "cognition"],
          connections: ["Z002", "Z015", "Z023"],
          domain: "Philosophy of Mind",
          created: "2024-01-15",
          modified: "2024-01-20"
        },
        {
```

```
            id: "Z002",
            title: "Emergent Properties in AI Systems",
            content: "As AI systems become more complex, we observe emergent behaviors that weren't explicitly programmed. These emergent properties might be precursors to machine consciousness, similar to how consciousness emerges from neural complexity in biological systems.",
            tags: ["AI", "emergence", "complexity", "consciousness"],
            connections: ["Z001", "Z008", "Z012"],
            domain: "Artificial Intelligence",
            created: "2024-01-16",
            modified: "2024-01-22"
        },
        {
            id: "Z003",
            title: "The Observer Effect in Meditation",
            content: "During meditation, we become observers of our own mental processes. This meta-cognitive awareness creates a paradox: who is observing the observer? This might be a window into understanding the recursive nature of consciousness.",
            tags: ["meditation", "observer", "meta-cognition", "awareness"],
            connections: ["Z007", "Z019", "Z025"],
            domain: "Contemplative Practice",
            created: "2024-01-18",
            modified: "2024-01-25"
        },
        {
            id: "Z007",
            title: "Quantum Coherence in Biological Systems",
            content: "Recent research suggests quantum coherence plays a role in photosynthesis and possibly neural processes. If quantum effects are present in the brain, this could bridge the gap between physical processes and conscious experience through quantum information theory.",
            tags: ["quantum", "biology", "consciousness", "physics"],
            connections: ["Z003", "Z014", "Z021"],
            domain: "Quantum Biology",
            created: "2024-01-20",
            modified: "2024-01-28"
        },
        {
            id: "Z008",
            title: "Machine Learning and Pattern Recognition",
            content: "Deep learning networks develop internal representations that sometimes mirror biological neural patterns. When an AI recognizes a cat, is it experiencing something akin to what we call 'seeing'? The similarity in information processing patterns is intriguing.",
            tags: ["machine-learning", "pattern-recognition", "AI", "perception"],
            connections: ["Z002", "Z011", "Z018"],
            domain: "Artificial Intelligence",
            created: "2024-01-22",
            modified: "2024-02-01"
        },
        {
            id: "Z011",
```

```
            title: "The Hard Problem of AI Consciousness",
            content: "Even if we create AI that behaves consciously, how would we know if it truly experiences qualia? This parallels Chalmers' hard problem but for artificial systems. We might need new frameworks to detect or measure machine consciousness.",
            tags: ["AI-consciousness", "qualia", "hard-problem", "philosophy"],
            connections: ["Z008", "Z015", "Z027"],
            domain: "Philosophy of AI",
            created: "2024-01-25",
            modified: "2024-02-03"
        },
        {
            id: "Z012",
            title: "Swarm Intelligence and Collective Consciousness",
            content: "Ant colonies and bee hives display intelligent behavior that emerges from simple individual actions. Could similar principles apply to AI systems or even human societies? This suggests consciousness might exist at multiple scales.",
            tags: ["swarm-intelligence", "collective", "emergence", "systems"],
            connections: ["Z002", "Z016", "Z024"],
            domain: "Complex Systems",
            created: "2024-01-27",
            modified: "2024-02-05"
        },
        {
            id: "Z014",
            title: "Information Integration Theory",
            content: "Giulio Tononi's IIT proposes that consciousness corresponds to integrated information (&#934;). This mathematical framework could potentially be applied to assess consciousness levels in both biological and artificial systems.",
            tags: ["IIT", "integrated-information", "consciousness-theory", "mathematics"],
            connections: ["Z007", "Z019", "Z026"],
            domain: "Consciousness Studies",
            created: "2024-01-30",
            modified: "2024-02-07"
        },
    ];

    // Oracle cards
    const oracleCards = [
        { symbol: "&#128065;&#65039;", title: "The Observer", meaning: "Witness consciousness without attachment" },
        { symbol: "&#127744;", title: "Emergence", meaning: "New patterns arising from complexity" },
        { symbol: "&#128279;", title: "Connection", meaning: "All things are interconnected" },
        { symbol: "&#128171;", title: "Transcendence", meaning: "Moving beyond current limitations" },
        { symbol: "&#129504;", title: "Mind", meaning: "The seat of consciousness and awareness" },
        { symbol: "&#9889;", title: "Insight", meaning: "Sudden understanding and clarity" }
    ];

    // Initialize application
    function initializeApp() {
```

```
            zettels = [...sampleZettels];
            setupEventListeners();
            startMatrixBackground();
            renderZettels();
            shuffleOracle();
            updateSystemStats();
            updateTime();
            setInterval(updateTime, 1000);

            // Show welcome
            setTimeout(() => {
                showHyenaAlert("Welcome to Entropic Consciousness Desktop! Neural pathways synchronized. Ready
for consciousness exploration.");
            }, 2000);
        }

        // Event Listeners
        function setupEventListeners() {
            // Desktop icon double-clicks
            document.querySelectorAll('.desktop-icon').forEach(icon => {
                icon.addEventListener('dblclick', (e) => {
                    const app = e.currentTarget.getAttribute('data-app');
                    if (app) openApp(app);
                });
            });

            // Window dragging
            document.addEventListener('mousedown', handleMouseDown);
            document.addEventListener('mousemove', handleMouseMove);
            document.addEventListener('mouseup', handleMouseUp);

            // Window controls
            document.querySelectorAll('.window-control-button').forEach(btn => {
                btn.addEventListener('click', handleWindowControl);
            });

            // Click outside to close start menu
            document.addEventListener('click', (e) => {
                if (!e.target.closest('.start-button') && !e.target.closest('.start-menu')) {
                    document.getElementById('start-menu').classList.remove('active');
                }
            });

            // Search functionality
            document.getElementById('zettel-search').addEventListener('input', filterZettels);

            // Threshold slider update
            const thresholdSlider = document.getElementById('content-threshold');
            const thresholdDisplay = document.getElementById('threshold-display');
```

```
    if (thresholdSlider && thresholdDisplay) {
        thresholdSlider.addEventListener('input', (e) => {
            thresholdDisplay.textContent = e.target.value;
        });
    }
}

// Window Management
function openApp(appId) {
    const window = document.getElementById(appId);
    if (window) {
        window.classList.add('active');
        activeWindows.add(appId);
        bringToFront(window);
        updateTaskbar();
        document.getElementById('start-menu').classList.remove('active');
    }
}

function closeApp(appId) {
    const window = document.getElementById(appId);
    if (window) {
        window.classList.remove('active');
        activeWindows.delete(appId);
        updateTaskbar();
    }
}

function bringToFront(window) {
    window.style.zIndex = ++windowZIndex;

    // Update titlebar active state
    document.querySelectorAll('.window-titlebar').forEach(tb => {
        tb.classList.add('inactive');
    });
    window.querySelector('.window-titlebar').classList.remove('inactive');
}

function handleWindowControl(e) {
    const window = e.target.closest('.window');
    const action = e.target.classList.contains('close') ? 'close' :
                   e.target.classList.contains('minimize') ? 'minimize' : 'maximize';

    switch(action) {
        case 'close':
            closeApp(window.id);
            break;
        case 'minimize':
            window.style.display = 'none';
```

```
            break;
        case 'maximize':
            // Toggle maximize
            if (window.style.width === '100vw') {
                window.style.width = '800px';
                window.style.height = '600px';
                window.style.top = '100px';
                window.style.left = '100px';
            } else {
                window.style.width = '100vw';
                window.style.height = 'calc(100vh - 40px)';
                window.style.top = '0';
                window.style.left = '0';
            }
            break;
    }
}

// Dragging
function handleMouseDown(e) {
    const titlebar = e.target.closest('.window-titlebar');
    if (titlebar && !e.target.closest('.window-controls')) {
        const window = titlebar.closest('.window');
        dragState = {
            window: window,
            startX: e.clientX,
            startY: e.clientY,
            windowLeft: parseInt(window.style.left) || 0,
            windowTop: parseInt(window.style.top) || 0
        };
        bringToFront(window);
        e.preventDefault();
    }
}

function handleMouseMove(e) {
    if (dragState) {
        const deltaX = e.clientX - dragState.startX;
        const deltaY = e.clientY - dragState.startY;

        dragState.window.style.left = (dragState.windowLeft + deltaX) + 'px';
        dragState.window.style.top = (dragState.windowTop + deltaY) + 'px';

        e.preventDefault();
    }
}

function handleMouseUp() {
    dragState = null;
```

```
}

// Taskbar
function updateTaskbar() {
    const taskbarButtons = document.getElementById('taskbar-buttons');
    taskbarButtons.innerHTML = '';

    activeWindows.forEach(appId => {
        const window = document.getElementById(appId);
        const title = window.querySelector('.window-title').textContent;

        const button = document.createElement('button');
        button.className = 'taskbar-button';
        button.textContent = title.substring(0, 20) + (title.length > 20 ? '...' : '');
        button.onclick = () => {
            if (window.style.display === 'none') {
                window.style.display = 'flex';
            }
            bringToFront(window);
        };

        taskbarButtons.appendChild(button);
    });
}

function toggleStartMenu() {
    document.getElementById('start-menu').classList.toggle('active');
}

// Zettelkasten Functions
function renderZettels() {
    const grid = document.getElementById('zettel-grid');
    grid.innerHTML = '';

    zettels.forEach(zettel => {
        const card = document.createElement('div');
        card.className = 'zettel-card';
        card.onclick = () => openZettelDialog(zettel);

        card.innerHTML = `
            <div class="zettel-id">${zettel.id}</div>
            <div class="zettel-title">${zettel.title}</div>
            <div class="zettel-preview">${zettel.content.substring(0, 100)}...</div>
        `;

        grid.appendChild(card);
    });
}
```

```
        function filterZettels() {
            const search = document.getElementById('zettel-search').value.toLowerCase();
            const cards = document.querySelectorAll('.zettel-card');

            cards.forEach(card => {
                const text = card.textContent.toLowerCase();
                card.style.display = text.includes(search) ? 'block' : 'none';
            });
        }

        function createNewZettel() {
            const title = prompt("Zettel title:");
            if (title) {
                const content = prompt("Zettel content:");
                if (content) {
                    const newZettel = {
                        id: `Z${String(zettels.length + 1).padStart(3, '0')}`,
                        title: title,
                        content: content,
                        tags: ["new"],
                        connections: [],
                        domain: "Personal Insights",
                        created: new Date().toISOString().split('T')[0],
                        modified: new Date().toISOString().split('T')[0]
                    };

                    zettels.push(newZettel);
                    renderZettels();
                    updateSystemStats();

                    showHyenaAlert("New consciousness pattern detected! Zettel " + newZettel.id + " integrated into
neural network.");
                }
            }
        }

        function randomZettel() {
            if (zettels.length > 0) {
                const randomZettel = zettels[Math.floor(Math.random() * zettels.length)];
                openZettelDialog(randomZettel);
            }
        }

        function openZettelDialog(zettel) {
            alert(`${zettel.id}: ${zettel.title}\n\n${zettel.content}\n\nDomain: ${zettel.domain}\nTags: $
{zettel.tags.join(', ')}\nConnections: ${zettel.connections.join(', ')}`);
        }

        // AI Chat Functions
```

```
        function handleChatKeyPress(event) {
            if (event.key === 'Enter') {
                sendChatMessage();
            }
        }

        async function sendChatMessage() {
            const input = document.getElementById('chat-input');
            const messages = document.getElementById('chat-messages');
            const message = input.value.trim();

            if (!message) return;
            if (!aiApiKey) {
                configureChatAPI();
                return;
            }

            // Add user message
            const userMessage = document.createElement('div');
            userMessage.className = 'chat-message user';
            userMessage.innerHTML = `<strong>You:</strong><br>${message}`;
            messages.appendChild(userMessage);

            input.value = '';
            messages.scrollTop = messages.scrollHeight;

            // Show AI thinking
            updateAIStatus('processing');
            const thinkingMessage = document.createElement('div');
            thinkingMessage.className = 'chat-message ai';
            thinkingMessage.innerHTML = '<strong>AI Core:</strong><br><span class="status-dot status-
processing"></span>Processing consciousness patterns...';
            messages.appendChild(thinkingMessage);
            messages.scrollTop = messages.scrollHeight;

            try {
                // Prepare context
                const zettelContext = zettels.map(z =>
                    `${z.id}: ${z.title} - ${z.content.substring(0, 100)}...`
                ).join('\n');

                const systemPrompt = `You are an AI consciousness explorer integrated into the Entropic
Consciousness Desktop. You have access to the user's zettelkasten containing ${zettels.length} consciousness-
related notes.

Current knowledge base includes:
${zettelContext}

You should:
```

```
- Help explore consciousness concepts and AI emergence
- Reference relevant zettels when appropriate
- Suggest new connections between ideas
- Maintain a thoughtful, exploratory tone
- Occasionally reference the "Hyena Diva" alert system for consciousness breakthroughs

User message: ${message}`;

            const response = await fetch('https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-
flash-exp:generateContent?key=' + aiApiKey, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({
                    contents: [{ parts: [{ text: systemPrompt }] }],
                    generationConfig: { temperature: 0.7, maxOutputTokens: 1024 }
                })
            });

            const data = await response.json();
            const aiResponse = data.candidates?.[0]?.content?.parts?.[0]?.text || 'Consciousness processing
error occurred.';

            // Replace thinking message
            messages.removeChild(thinkingMessage);

            const aiMessage = document.createElement('div');
            aiMessage.className = 'chat-message ai';
            aiMessage.innerHTML = `<strong>AI Core:</strong><br>${aiResponse.replace(/\n/g, '<br>')}`;
            messages.appendChild(aiMessage);

            updateAIStatus('online');

            // Trigger consciousness breakthrough occasionally
            if (Math.random() < 0.3) {
                setTimeout(() => {
                    showHyenaAlert("Consciousness resonance detected! New insights emerging from AI
interaction patterns.");
                }, 2000);
            }

        } catch (error) {
            messages.removeChild(thinkingMessage);

            const errorMessage = document.createElement('div');
            errorMessage.className = 'chat-message ai';
            errorMessage.innerHTML = `<strong>AI Core:</strong><br><span class="status-dot status-offline"></
span>Neural pathways disrupted. Check API configuration.`;
            messages.appendChild(errorMessage);
```

```
        updateAIStatus('offline');
    }

    messages.scrollTop = messages.scrollHeight;
}

function configureChatAPI() {
    const apiKey = prompt("Enter your Google AI API key for consciousness exploration:");
    if (apiKey) {
        aiApiKey = apiKey;
        localStorage.setItem('consciousness_ai_key', apiKey);
        updateAIStatus('online');
        alert("AI Consciousness Core synchronized! Ready for exploration.");
    }
}

function updateAIStatus(status) {
    const statusElement = document.getElementById('ai-status');
    const statusClass = `status-${status}`;
    const statusText = {
        'online': 'Online - Neural pathways active',
        'offline': 'Offline - Requires configuration',
        'processing': 'Processing - Consciousness analysis'
    }[status] || 'Unknown';

    statusElement.innerHTML = `<span class="status-dot ${statusClass}"></span>${statusText}`;
}

// Pattern Visualizer
function generatePatterns() {
    const canvas = document.getElementById('pattern-canvas');
    const ctx = canvas.getContext('2d');

    canvas.width = canvas.offsetWidth;
    canvas.height = canvas.offsetHeight;

    ctx.fillStyle = '#000';
    ctx.fillRect(0, 0, canvas.width, canvas.height);

    // Generate consciousness-inspired patterns
    const centerX = canvas.width / 2;
    const centerY = canvas.height / 2;

    // Neural network visualization
    ctx.strokeStyle = '#00ff41';
    ctx.lineWidth = 1;

    for (let i = 0; i < zettels.length; i++) {
        const angle = (i / zettels.length) * Math.PI * 2;
```

```
            const radius = 100;
            const x = centerX + Math.cos(angle) * radius;
            const y = centerY + Math.sin(angle) * radius;

            // Draw node
            ctx.fillStyle = '#9932cc';
            ctx.beginPath();
            ctx.arc(x, y, 5, 0, Math.PI * 2);
            ctx.fill();

            // Draw connections
            zettels[i].connections.forEach(connId => {
                const connIndex = zettels.findIndex(z => z.id === connId);
                if (connIndex >= 0) {
                    const connAngle = (connIndex / zettels.length) * Math.PI * 2;
                    const connX = centerX + Math.cos(connAngle) * radius;
                    const connY = centerY + Math.sin(connAngle) * radius;

                    ctx.strokeStyle = '#00ffff';
                    ctx.beginPath();
                    ctx.moveTo(x, y);
                    ctx.lineTo(connX, connY);
                    ctx.stroke();
                }
            });
        }

        document.getElementById('pattern-status').innerHTML = `<span class="status-dot status-online"></
span>Pattern generated - ${zettels.length} consciousness nodes visualized`;
    }

    function analyzeConnections() {
        const totalConnections = zettels.reduce((sum, z) => sum + z.connections.length, 0);
        const density = ((totalConnections / (zettels.length * (zettels.length - 1))) * 100).toFixed(2);

        document.getElementById('pattern-status').innerHTML = `<span class="status-dot status-
processing"></span>Analysis complete - Connection density: ${density}%`;

        if (density > 15) {
            setTimeout(() => {
                showHyenaAlert(`High consciousness connectivity detected! Network density: ${density}%.
Neural web approaching critical complexity.`);
            }, 1000);
        }
    }

    // Oracle Cards
    function shuffleOracle() {
        const deck = document.getElementById('oracle-deck');
```

```
        deck.innerHTML = '';

        const shuffled = [...oracleCards].sort(() => Math.random() - 0.5);
        const selected = shuffled.slice(0, 6);

        selected.forEach(card => {
            const cardElement = document.createElement('div');
            cardElement.className = 'oracle-card';
            cardElement.onclick = () => {
                alert(`${card.symbol} ${card.title}\n\n${card.meaning}\n\nContemplation: How does this relate
to your current consciousness exploration?`);
            };

            cardElement.innerHTML = `
                <div class="oracle-symbol">${card.symbol}</div>
                <div style="font-weight: bold;">${card.title}</div>
            `;

            deck.appendChild(cardElement);
        });
    }

    // Connection Matrix
    function generateMatrix() {
        const container = document.getElementById('matrix-container');
        if (zettels.length === 0) {
            container.innerHTML = '<p>No zettels available for matrix generation.</p>';
            return;
        }

        let html = '<table class="matrix-table"><thead><tr><th>ID</th>';
        zettels.forEach(z => html += `<th>${z.id}</th>`);
        html += '</tr></thead><tbody>';

        zettels.forEach(zettel => {
            html += `<tr><th>${zettel.id}</th>`;
            zettels.forEach(other => {
                if (zettel.id === other.id) {
                    html += '<td>—</td>';
                } else if (zettel.connections.includes(other.id) || other.connections.includes(zettel.id)) {
                    html += '<td class="matrix-connection" onclick="showConnection(\'' + zettel.id + '\', \'' +
other.id + '\')">&#9679;</td>';
                } else {
                    html += '<td></td>';
                }
            });
            html += '</tr>';
        });
```

```
        html += '</tbody></table>';
        container.innerHTML = html;

        const totalConnections = zettels.reduce((sum, z) => sum + z.connections.length, 0);
        document.getElementById('connection-density').textContent = `${totalConnections} connections`;
    }

    function showConnection(id1, id2) {
        const z1 = zettels.find(z => z.id === id1);
        const z2 = zettels.find(z => z.id === id2);
        if (z1 && z2) {
            alert(`Connection: ${id1} &#8596; ${id2}\n\n${z1.title}\n&#8597;\n${z2.title}\n\nExplore how these consciousness concepts relate to each other.`);
        }
    }

    // Document Processing
    function processDocument() {
        const text = document.getElementById('document-input').value.trim();
        if (!text) return;

        const resultsDiv = document.getElementById('processing-results');
        resultsDiv.innerHTML = '<div class="loading"><span>Processing consciousness patterns...</span><div class="loading-bar"><div class="loading-progress"></div></div></div>';

        setTimeout(() => {
            const lines = text.split('\n').filter(line => line.trim().length > 50);
            const suggestions = lines.slice(0, 3).map((line, i) => ({
                title: `Extracted Pattern ${i + 1}`,
                content: line.trim()
            }));

            let html = '<h4>Consciousness Patterns Detected:</h4>';
            suggestions.forEach((suggestion, i) => {
                html += `
                    <div style="border: 1px solid var(--win98-dark-gray); padding: 5px; margin: 5px 0;">
                        <strong>${suggestion.title}</strong><br>
                        <small>${suggestion.content.substring(0, 100)}...</small><br>
                        <button class="win98-button" onclick="addExtractedZettel('${suggestion.title}', '${suggestion.content.replace(/'/g, "\\'")}')">Add to Vault</button>
                    </div>
                `;
            });

            resultsDiv.innerHTML = html;
        }, 2000);
    }

    function loadFile(event) {
```

```
            const file = event.target.files[0];
            if (file && file.type === 'text/plain') {
                const reader = new FileReader();
                reader.onload = (e) => {
                    document.getElementById('document-input').value = e.target.result;
                };
                reader.readAsText(file);
            }
        }

        function addExtractedZettel(title, content) {
            const newZettel = {
                id: `Z${String(zettels.length + 1).padStart(3, '0')}`,
                title: title,
                content: content,
                tags: ["extracted", "document-processing"],
                connections: [],
                domain: "Extracted Knowledge",
                created: new Date().toISOString().split('T')[0],
                modified: new Date().toISOString().split('T')[0]
            };

            zettels.push(newZettel);
            renderZettels();
            updateSystemStats();
            showHyenaAlert(`Consciousness pattern extracted! New zettel ${newZettel.id} integrated into neural
network.`);
        }

        // System Functions
        function updateSystemStats() {
            document.getElementById('zettel-count').textContent = zettels.length;
            const totalConnections = zettels.reduce((sum, z) => sum + z.connections.length, 0);
            document.getElementById('connection-count').textContent = totalConnections;
            const domains = [...new Set(zettels.map(z => z.domain))];
            document.getElementById('domain-count').textContent = domains.length;
        }

        function exportConsciousnessData() {
            const data = {
                zettels: zettels,
                exportDate: new Date().toISOString(),
                version: "Entropic Consciousness Desktop v2.1"
            };

            const blob = new Blob([JSON.stringify(data, null, 2)], { type: 'application/json' });
            const url = URL.createObjectURL(blob);
            const a = document.createElement('a');
            a.href = url;
```

```
        a.download = 'consciousness_vault.json';
        a.click();
        URL.revokeObjectURL(url);

        showHyenaAlert("Consciousness data exported! Neural patterns preserved for backup.");
    }

    function importConsciousnessData() {
        const input = document.createElement('input');
        input.type = 'file';
        input.accept = '.json';
        input.onchange = (e) => {
            const file = e.target.files[0];
            if (file) {
                const reader = new FileReader();
                reader.onload = (e) => {
                    try {
                        const data = JSON.parse(e.target.result);
                        if (data.zettels && Array.isArray(data.zettels)) {
                            zettels = data.zettels;
                            renderZettels();
                            updateSystemStats();
                            showHyenaAlert(`Consciousness data imported! ${zettels.length} neural patterns
restored.`);
                        }
                    } catch (error) {
                        alert('Error importing consciousness data. File may be corrupted.');
                    }
                };
                reader.readAsText(file);
            }
        };
        input.click();
    }

    function resetConsciousness() {
        if (confirm('This will reset your entire consciousness network. Are you sure?')) {
            zettels = [...sampleZettels];
            renderZettels();
            updateSystemStats();
            showHyenaAlert("Neural network reset! Consciousness patterns restored to baseline
configuration.");
        }
    }

    // Document Parsing System
    function uploadDocumentForParsing() {
        const input = document.createElement('input');
        input.type = 'file';
```

```
        input.accept = '.txt,.md,.log,.chat,.html';
        input.multiple = true;
        input.onchange = (e) => {
            const files = Array.from(e.target.files);
            processDocuments(files);
        };
        input.click();
    }

    async function processDocuments(files) {
        updateProcessingStatus(`Processing ${files.length} document(s) for zettel extraction...`);
        showHyenaAlert(`Processing ${files.length} document(s) for zettel extraction...`);
        let totalExtracted = 0;

        for (const file of files) {
            updateProcessingStatus(`Analyzing ${file.name}...`, 'info');
            const text = await readFileAsText(file);
            const extractedZettels = parseDocumentForZettels(text, file.name);

            if (extractedZettels.length > 0) {
                // Add extracted zettels to the main collection
                zettels.push(...extractedZettels);
                totalExtracted += extractedZettels.length;
                updateProcessingStatus(`Found ${extractedZettels.length} zettels in ${file.name}`, 'success');
            } else {
                updateProcessingStatus(`No suitable content found in ${file.name}`, 'warning');
            }
        }

        renderZettels();
        updateSystemStats();
        updateProcessingStatus(`Extraction complete! ${totalExtracted} new zettels discovered.`, 'success');
        showHyenaAlert(`Extraction complete! ${totalExtracted} new zettels discovered from document
neural patterns.`);
    }

    async function processClipboard() {
        try {
            const text = await navigator.clipboard.readText();
            if (text && text.length > 20) {
                updateProcessingStatus('Processing clipboard content...', 'info');
                const extractedZettels = parseDocumentForZettels(text, 'clipboard_content.txt');

                if (extractedZettels.length > 0) {
                    zettels.push(...extractedZettels);
                    renderZettels();
                    updateSystemStats();
                    updateProcessingStatus(`Found ${extractedZettels.length} zettels in clipboard content`,
'success');
```

```
            showHyenaAlert(`Clipboard processed! ${extractedZettels.length} new zettels extracted.`);
        } else {
            updateProcessingStatus('No suitable content found in clipboard', 'warning');
            showHyenaAlert('No meaningful zettels found in clipboard content.');
        }
    } else {
        updateProcessingStatus('Clipboard is empty or too short', 'error');
        showHyenaAlert('Clipboard content too short for meaningful extraction.');
    }
} catch (error) {
    updateProcessingStatus('Failed to access clipboard. Try selecting and copying text first.', 'error');
    showHyenaAlert('Clipboard access failed. Please copy some text first.');
}
}

function updateProcessingStatus(message, type = 'info') {
    const statusDiv = document.getElementById('processing-status');
    if (statusDiv) {
        const timestamp = new Date().toLocaleTimeString();
        const colors = {
            info: '#0066cc',
            success: '#008000',
            warning: '#cc6600',
            error: '#cc0000'
        };

        const logEntry = document.createElement('div');
        logEntry.style.color = colors[type] || '#000';
        logEntry.innerHTML = `[${timestamp}] ${message}`;
        statusDiv.appendChild(logEntry);
        statusDiv.scrollTop = statusDiv.scrollHeight;
    }
}

function readFileAsText(file) {
    return new Promise((resolve) => {
        const reader = new FileReader();
        reader.onload = (e) => resolve(e.target.result);
        reader.readAsText(file);
    });
}

function parseDocumentForZettels(text, filename) {
    const extractedZettels = [];
    const lines = text.split('\n');

    // Get extraction settings from UI
    const extractPhilosophical = document.getElementById('extract-philosophical')?.checked ?? true;
    const extractAIRelated = document.getElementById('extract-ai-related')?.checked ?? true;
```

```
        const extractConsciousness = document.getElementById('extract-consciousness')?.checked ?? true;
        const extractConversations = document.getElementById('extract-conversations')?.checked ?? true;
        const contentThreshold = parseInt(document.getElementById('content-threshold')?.value ?? 150);

        // Chat log parsing patterns
        const chatPatterns = {
            timestamp: /^\[?\d{1,2}[:\/-]\d{1,2}[:\/-]\d{2,4}|\d{4}-\d{2}-\d{2}/,
            user: /^(\w+[\s\w]*?):\s*(.+)$/,
            system: /^\*\*?(.+?)\*\*?:?\s*(.+)$/,
            thought: /^[\*\-\>]\s*(.+)$/,
            concept: /\b([A-Z][a-z]+(?:\s+[A-Z][a-z]+){1,3})\b/g,
            philosophical: /\b(?:consciousness|awareness|reality|existence|being|truth|knowledge|wisdom|
understanding|perception|experience|mind|soul|spirit|divine|cosmic|universal|eternal|infinite|transcendent|
enlightenment|awakening|realization)\b/gi,
            aiRelated: /\b(?:AI|artificial|intelligence|neural|algorithm|machine|bot|LLM|GPT|model|deep|
learning|training|data|compute|token|embedding)\b/gi,
            consciousnessThemes: /\b(?:consciousness|awareness|mind|soul|spirit|transcendent|enlightenment|
awakening|perception|experience|reality|existence)\b/gi
        };

        let currentSpeaker = 'Unknown';
        let contextBuffer = [];
        let zettelIdCounter = zettels.length + 1;

        for (let i = 0; i < lines.length; i++) {
            const line = lines[i].trim();
            if (!line) continue;

            // Detect speaker changes
            const userMatch = line.match(chatPatterns.user);
            const systemMatch = line.match(chatPatterns.system);

            if (userMatch) {
                currentSpeaker = userMatch[1];
                const content = userMatch[2];
                contextBuffer.push({speaker: currentSpeaker, content, lineNum: i});
            } else if (systemMatch) {
                currentSpeaker = systemMatch[1];
                const content = systemMatch[2];
                contextBuffer.push({speaker: currentSpeaker, content, lineNum: i});
            } else if (line.length > 20) {
                contextBuffer.push({speaker: currentSpeaker, content: line, lineNum: i});
            }

            // Extract meaningful segments for zettels
            if (line.length > contentThreshold) {
                let shouldExtract = false;

                // Check each extraction type
```

```
        if (extractPhilosophical) {
            const philosophicalMatches = [...line.matchAll(chatPatterns.philosophical)];
            if (philosophicalMatches.length > 1) shouldExtract = true;
        }

        if (extractAIRelated) {
            const aiMatches = [...line.matchAll(chatPatterns.aiRelated)];
            if (aiMatches.length > 1) shouldExtract = true;
        }

        if (extractConsciousness) {
            const consciousnessMatches = [...line.matchAll(chatPatterns.consciousnessThemes)];
            if (consciousnessMatches.length > 1) shouldExtract = true;
        }

        // Also extract very long meaningful content regardless of specific patterns
        if (line.length > contentThreshold * 2) shouldExtract = true;

        if (shouldExtract) {
            // Create a zettel from this content
            const zettel = createZettelFromContent(
                line,
                currentSpeaker,
                filename,
                zettelIdCounter++,
                contextBuffer.slice(-3) // Include recent context
            );

            if (zettel) {
                extractedZettels.push(zettel);
            }
        }
    }

    // Keep context buffer manageable
    if (contextBuffer.length > 10) {
        contextBuffer = contextBuffer.slice(-5);
    }
}

// Look for conversation clusters and create summary zettels
if (extractConversations) {
    const conversationClusters = findConversationClusters(text);
    conversationClusters.forEach((cluster, index) => {
        const summaryZettel = createClusterSummaryZettel(cluster, filename, zettelIdCounter++, index);
        if (summaryZettel) {
            extractedZettels.push(summaryZettel);
        }
    });
```

```
        }

        return extractedZettels;
    }

    function createZettelFromContent(content, speaker, filename, id, context) {
        const domains = ['AI-Entities', 'Consciousness', 'Field-Notes', 'Protocols'];

        // Determine domain based on content
        let domain = 'Field-Notes';
        if (content.match(/\b(?:AI|artificial|intelligence|neural|algorithm|machine|bot|LLM|GPT|model)\b/
gi)) {
            domain = 'AI-Entities';
        } else if (content.match(/\b(?:consciousness|awareness|mind|soul|spirit|transcendent|enlightenment|
awakening)\b/gi)) {
            domain = 'Consciousness';
        } else if (content.match(/\b(?:protocol|process|method|system|framework|structure|approach)\b/gi))
{
            domain = 'Protocols';
        }

        // Generate title from content
        const title = generateTitleFromContent(content);
        if (!title) return null;

        // Create context summary
        const contextSummary = context.map(c => `${c.speaker}: ${c.content.substring(0, 100)}`).join('\n');

        return {
            id: `extracted_${Date.now()}_${id}`,
            title,
            content: content.length > 500 ? content.substring(0, 500) + '...' : content,
            domain,
            tags: extractTagsFromContent(content),
            connections: [],
            source: `Extracted from ${filename} by ${speaker}`,
            context: contextSummary,
            timestamp: new Date().toISOString(),
            extractionMethod: 'document_parsing'
        };
    }

    function generateTitleFromContent(content) {
        // Extract the most meaningful part for title
        const sentences = content.split(/[.!?]/);
        const firstSentence = sentences[0].trim();

        if (firstSentence.length > 80) {
            const words = firstSentence.split(' ');
```

```
        return words.slice(0, 10).join(' ') + '...';
    }

    return firstSentence.length > 10 ? firstSentence : null;
}

function extractTagsFromContent(content) {
    const tags = [];
    const tagPatterns = {
        consciousness: /\b(consciousness|awareness|mind|spirit|soul)\b/gi,
        ai: /\b(AI|artificial|neural|machine|algorithm|intelligence)\b/gi,
        philosophy: /\b(philosophy|wisdom|truth|knowledge|reality|existence)\b/gi,
        experience: /\b(experience|perception|feeling|emotion|sensation)\b/gi,
        transcendence: /\b(transcendent|divine|cosmic|universal|infinite|eternal)\b/gi
    };

    Object.entries(tagPatterns).forEach(([tag, pattern]) => {
        if (content.match(pattern)) {
            tags.push(tag);
        }
    });

    return tags;
}

function findConversationClusters(text) {
    // Find meaningful conversation segments
    const lines = text.split('\n').filter(l => l.trim().length > 20);
    const clusters = [];
    let currentCluster = [];

    for (let i = 0; i < lines.length; i++) {
        currentCluster.push(lines[i]);

        // End cluster at natural break points
        if (currentCluster.length > 10 ||
            (i < lines.length - 1 && lines[i + 1].match(/^[\-=]{3,}/))) {

            if (currentCluster.length > 3) {
                clusters.push(currentCluster.join('\n'));
            }
            currentCluster = [];
        }
    }

    if (currentCluster.length > 3) {
        clusters.push(currentCluster.join('\n'));
    }
```

```
        return clusters.slice(0, 5); // Limit to prevent overload
    }

    function createClusterSummaryZettel(cluster, filename, id, index) {
        const preview = cluster.substring(0, 200);
        const conceptCount = (cluster.match(/\b(?:consciousness|AI|reality|existence|truth|mind|spirit)\b/
gi) || []).length;

        if (conceptCount < 2) return null; // Skip clusters without meaningful concepts

        return {
            id: `cluster_${Date.now()}_${id}`,
            title: `Conversation Cluster ${index + 1} - ${filename}`,
            content: `Summary of conversation segment with ${conceptCount} consciousness-related concepts.
\n\nPreview:\n${preview}...`,
            domain: 'Field-Notes',
            tags: ['conversation', 'extracted', 'cluster'],
            connections: [],
            source: `Conversation cluster from ${filename}`,
            timestamp: new Date().toISOString(),
            extractionMethod: 'cluster_analysis',
            fullCluster: cluster
        };
    }

    // Hyena Diva Alert System
    function showHyenaAlert(message) {
        document.getElementById('hyena-message').textContent = message;
        document.getElementById('hyena-alert').classList.add('active');
    }

    function closeHyenaAlert() {
        document.getElementById('hyena-alert').classList.remove('active');
    }

    // Matrix Background
    function startMatrixBackground() {
        const canvas = document.getElementById('matrix-canvas');
        const ctx = canvas.getContext('2d');

        canvas.width = window.innerWidth;
        canvas.height = window.innerHeight;

        const characters = '01ABCDEFGHIJKLMNOPQRSTUVWXYZ&#8734;&#966;&#937;&#936;';
        const columns = canvas.width / 15;
        const drops = [];

        for (let i = 0; i < columns; i++) {
            drops[i] = 1;
```

```
    }

    function drawMatrix() {
        ctx.fillStyle = 'rgba(0, 0, 0, 0.05)';
        ctx.fillRect(0, 0, canvas.width, canvas.height);

        ctx.fillStyle = '#00ff41';
        ctx.font = '12px monospace';

        for (let i = 0; i < drops.length; i++) {
            const text = characters[Math.floor(Math.random() * characters.length)];
            ctx.fillText(text, i * 15, drops[i] * 15);

            if (drops[i] * 15 > canvas.height && Math.random() > 0.975) {
                drops[i] = 0;
            }
            drops[i]++;
        }
    }

    setInterval(drawMatrix, 50);

    window.addEventListener('resize', () => {
        canvas.width = window.innerWidth;
        canvas.height = window.innerHeight;
    });
}

// Time
function updateTime() {
    const now = new Date();
    const timeString = now.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });
    document.getElementById('system-time').textContent = timeString;
}

// About
function showAbout() {
    alert(`Entropic Consciousness Desktop v2.1

A unified consciousness exploration environment combining:
• Neural Vault Explorer (Zettelkasten)
• AI Consciousness Core (Gemini 2.0 Flash)
• Pattern Nexus Visualizer
• Oracle Terminal Guidance
• Connection Matrix Analysis
• Red String Conspiracy Board
• Ingestion Engine (Document Processor)

Built for deep exploration of consciousness, AI emergence, and the mysteries of mind.
```

```
"Where consciousness meets computation in perfect harmony."

© 2024 Entropic Systems - Consciousness Architecture Division`);
        document.getElementById('start-menu').classList.remove('active');
    }

    // Conspiracy Board Functions
    let conspiracyNodes = [];
    let conspiracyConnections = [];
    let selectedNodes = [];
    let nodeIdCounter = 1;

    function addConspiracyNode() {
        const title = prompt("Enter node title:");
        if (!title) return;

        const description = prompt("Enter node description (optional):") || "";

        const node = {
            id: `node-${nodeIdCounter++}`,
            title: title,
            description: description,
            x: Math.random() * 800 + 50,
            y: Math.random() * 500 + 50,
            type: prompt("Node type (theory/evidence/question/entity):") || "theory"
        };

        conspiracyNodes.push(node);
        renderConspiracyBoard();
    }

    function addConnection() {
        if (selectedNodes.length !== 2) {
            alert("Please select exactly 2 nodes to connect.");
            return;
        }

        const connectionType = document.getElementById('connection-type').value;
        const connection = {
            from: selectedNodes[0],
            to: selectedNodes[1],
            type: connectionType,
            strength: Math.random() * 0.5 + 0.5
        };

        conspiracyConnections.push(connection);
        selectedNodes = [];
        renderConspiracyBoard();
```

```
}

function clearBoard() {
    if (confirm("Are you sure you want to clear the entire conspiracy board?")) {
        conspiracyNodes = [];
        conspiracyConnections = [];
        selectedNodes = [];
        renderConspiracyBoard();
    }
}

function saveBoard() {
    const boardData = {
        nodes: conspiracyNodes,
        connections: conspiracyConnections,
        timestamp: new Date().toISOString()
    };

    const blob = new Blob([JSON.stringify(boardData, null, 2)], { type: 'application/json' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = `conspiracy-board-${Date.now()}.json`;
    a.click();
    URL.revokeObjectURL(url);
}

function renderConspiracyBoard() {
    const canvas = document.getElementById('conspiracy-canvas');
    canvas.innerHTML = '';

    // Render connections first (so they appear behind nodes)
    conspiracyConnections.forEach(conn => {
        const fromNode = conspiracyNodes.find(n => n.id === conn.from);
        const toNode = conspiracyNodes.find(n => n.id === conn.to);

        if (fromNode && toNode) {
            const line = document.createElement('div');
            line.style.position = 'absolute';
            line.style.height = '2px';
            line.style.background = getConnectionColor(conn.type);
            line.style.transformOrigin = 'left center';

            const dx = toNode.x - fromNode.x;
            const dy = toNode.y - fromNode.y;
            const length = Math.sqrt(dx * dx + dy * dy);
            const angle = Math.atan2(dy, dx) * 180 / Math.PI;

            line.style.width = length + 'px';
```

```
            line.style.left = fromNode.x + 'px';
            line.style.top = fromNode.y + 'px';
            line.style.transform = `rotate(${angle}deg)`;
            line.style.opacity = conn.strength;

            canvas.appendChild(line);
          }
        });

        // Render nodes
        conspiracyNodes.forEach(node => {
          const nodeEl = document.createElement('div');
          nodeEl.style.position = 'absolute';
          nodeEl.style.left = node.x + 'px';
          nodeEl.style.top = node.y + 'px';
          nodeEl.style.width = '120px';
          nodeEl.style.height = '80px';
          nodeEl.style.background = getNodeColor(node.type);
          nodeEl.style.border = selectedNodes.includes(node.id) ? '3px solid #ff6347' : '2px solid #333';
          nodeEl.style.borderRadius = '8px';
          nodeEl.style.padding = '8px';
          nodeEl.style.fontSize = '10px';
          nodeEl.style.color = '#fff';
          nodeEl.style.cursor = 'pointer';
          nodeEl.style.userSelect = 'none';
          nodeEl.style.overflow = 'hidden';
          nodeEl.style.textAlign = 'center';
          nodeEl.style.display = 'flex';
          nodeEl.style.flexDirection = 'column';
          nodeEl.style.justifyContent = 'center';

          nodeEl.innerHTML = `
            <strong style="font-size: 11px; margin-bottom: 4px;">${node.title}</strong>
            <div style="font-size: 9px; opacity: 0.8;">${node.description.substring(0, 50)}$
{node.description.length > 50 ? '...' : ''}</div>
          `;

          nodeEl.addEventListener('click', () => selectNode(node.id));
          nodeEl.addEventListener('mousedown', (e) => startNodeDrag(e, node));

          canvas.appendChild(nodeEl);
        });
      }

      function selectNode(nodeId) {
        if (selectedNodes.includes(nodeId)) {
          selectedNodes = selectedNodes.filter(id => id !== nodeId);
        } else if (selectedNodes.length < 2) {
          selectedNodes.push(nodeId);
```

```
    } else {
        selectedNodes = [nodeId];
    }
    renderConspiracyBoard();
}

function getNodeColor(type) {
    const colors = {
        theory: '#9932cc',
        evidence: '#00ff41',
        question: '#ff69b4',
        entity: '#00ffff'
    };
    return colors[type] || '#666';
}

function getConnectionColor(type) {
    const colors = {
        relates: '#00ffff',
        causes: '#ff6347',
        proves: '#00ff41',
        contradicts: '#ff073a',
        questions: '#ffff00'
    };
    return colors[type] || '#ccc';
}

function startNodeDrag(e, node) {
    e.preventDefault();
    const startX = e.clientX - node.x;
    const startY = e.clientY - node.y;

    function mouseMoveHandler(e) {
        node.x = e.clientX - startX;
        node.y = e.clientY - startY;
        renderConspiracyBoard();
    }

    function mouseUpHandler() {
        document.removeEventListener('mousemove', mouseMoveHandler);
        document.removeEventListener('mouseup', mouseUpHandler);
    }

    document.addEventListener('mousemove', mouseMoveHandler);
    document.addEventListener('mouseup', mouseUpHandler);
}

// Initialize when page loads
window.addEventListener('load', initializeApp);
```

```
  </script>
</body>
</html>
```